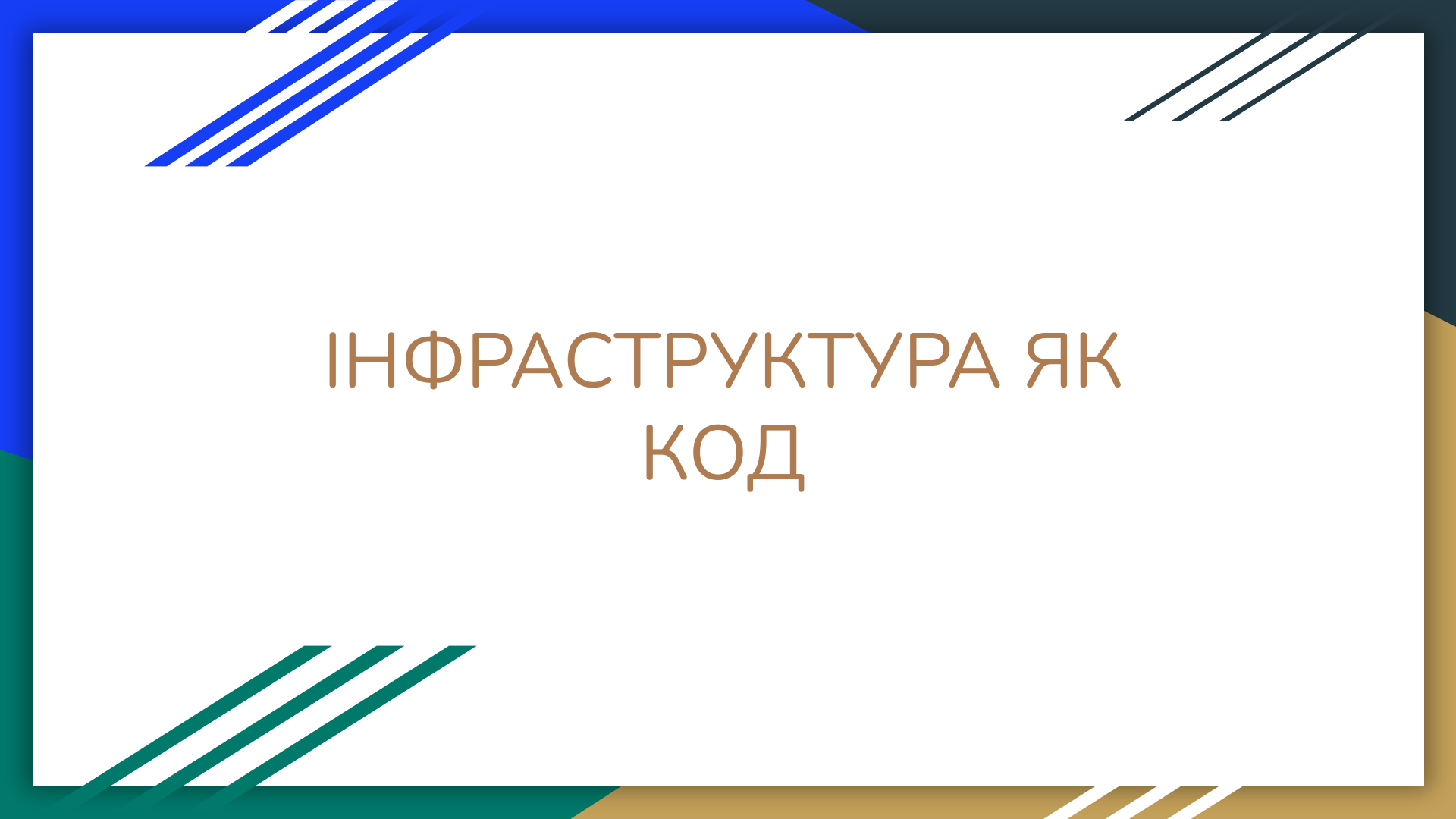




ІНФРАСТРУКТУРА ЯК КОД

НЕКОНТРОЛЬОВАНА АВТОМАТИЗАЦІЯ

Інструменти працюють за розкладом для застосування, повторного застосування та оновлення конфігурації

ПЕРЕВАГИ АВТОМАТИЗАЦІЇ:

- Швидко **виявляйте** проблеми
- Змусьте себе **вирішити** ці проблеми
- Змусьте себе **вдосконалити** свої інструменти та процеси
- Перешкоджає змінам «поза діапазоном».

АВТОМАТИЗУВАТИ ОНОВЛЕННЯ СЕРВЕРА

Автоматизація стосується не лише нових серверів!

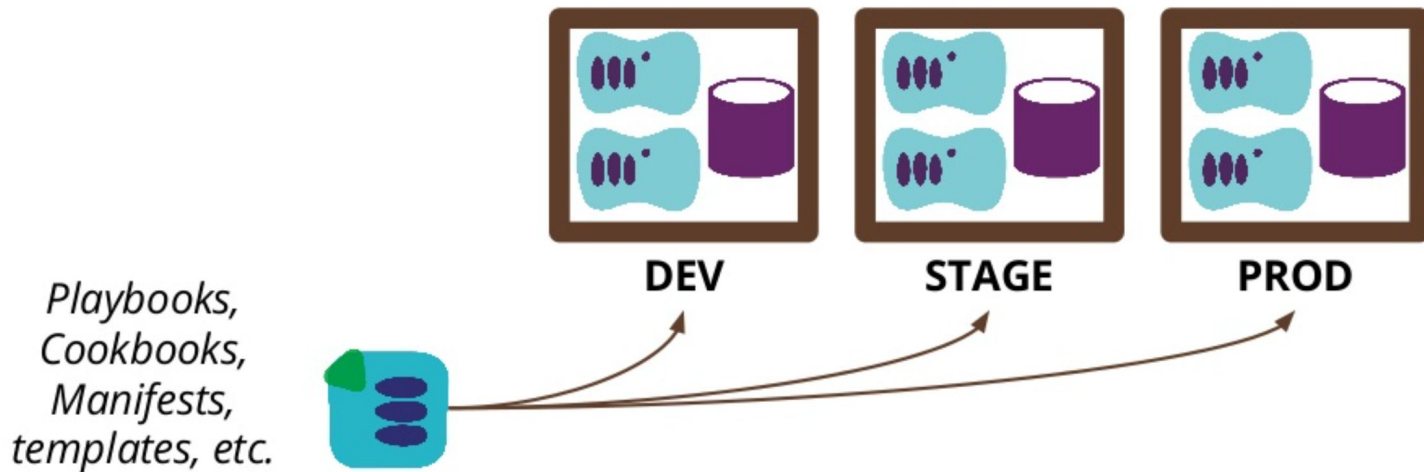
Синхронізація конфігурації. Запустіть Chef, Puppet, Ansible тощо за розкладом

Незмінні сервери Застосуйте зміни шляхом перебудови серверів

Контейнерні сервери Застосовуйте зміни, розгортаючи нові екземпляри контейнерів

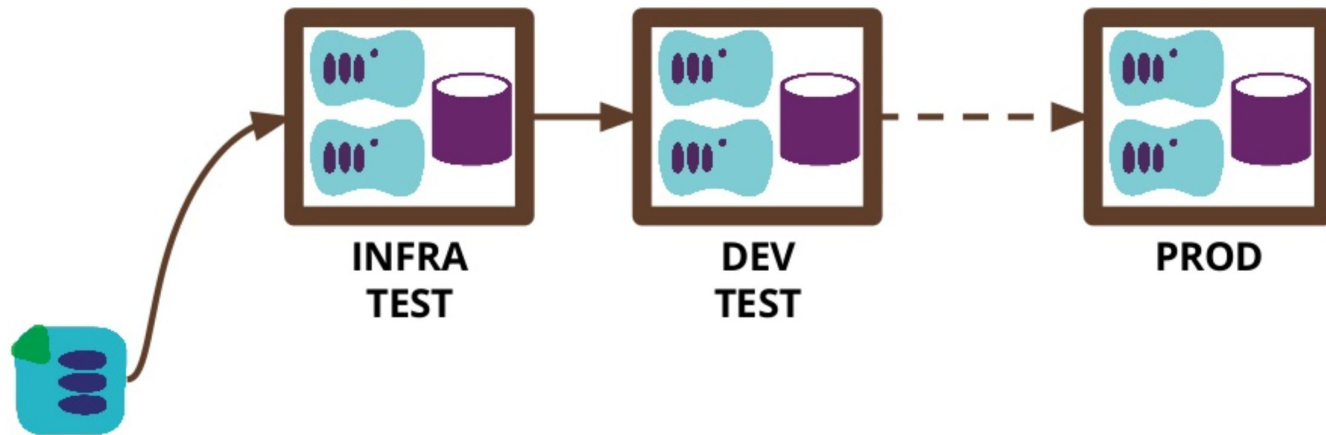
ПОВТОРНЕ ВИКОРИСТАННЯ та РОЗШИРЕННЯ ВИЗНАЧЕНЬ

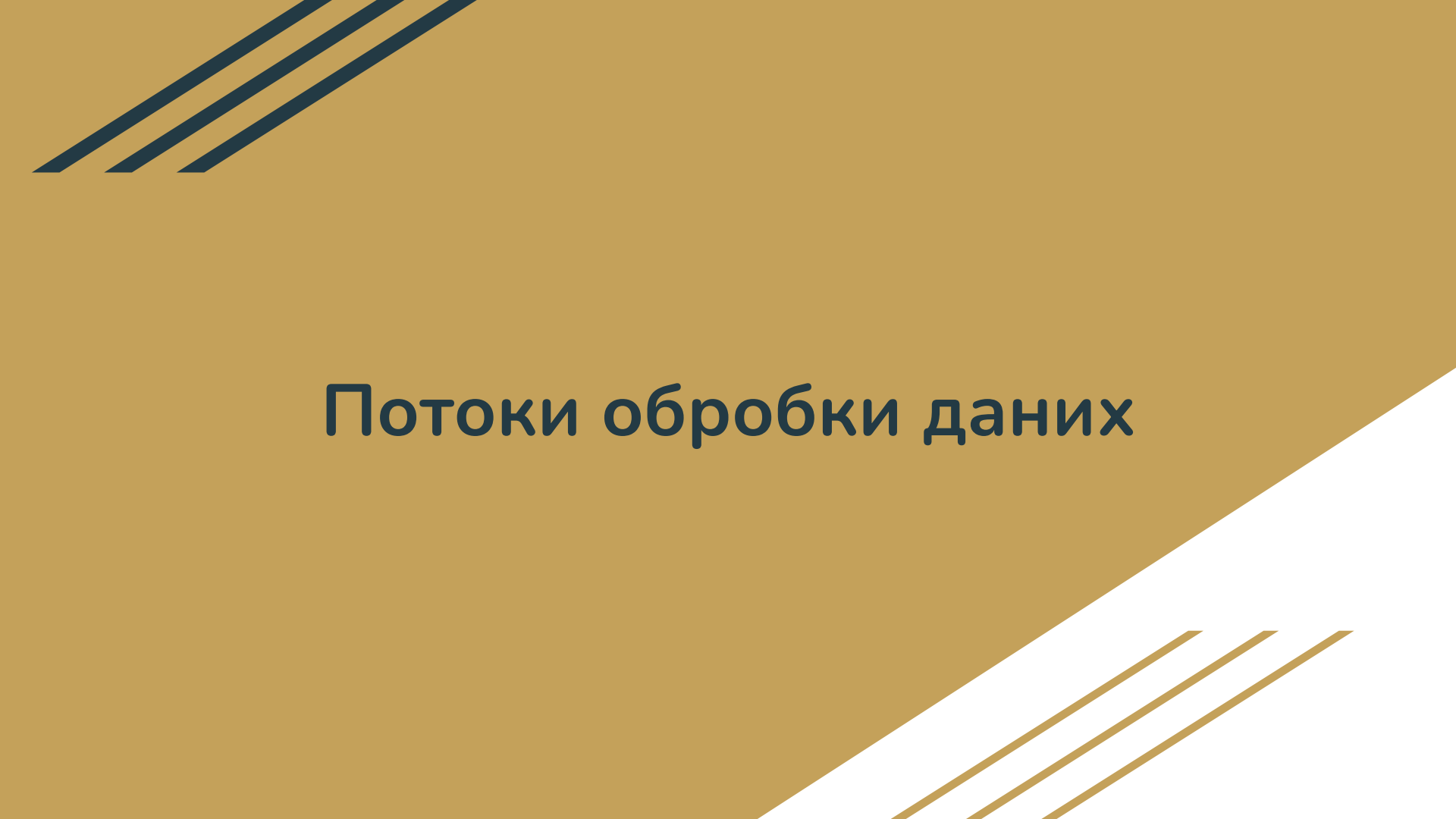
Повторне використання тих самих файлів визначення в різних середовищах для певної програми чи служби



ЗМІНИ ІНФРАСТРУКТУРИ ТЕСТІВ

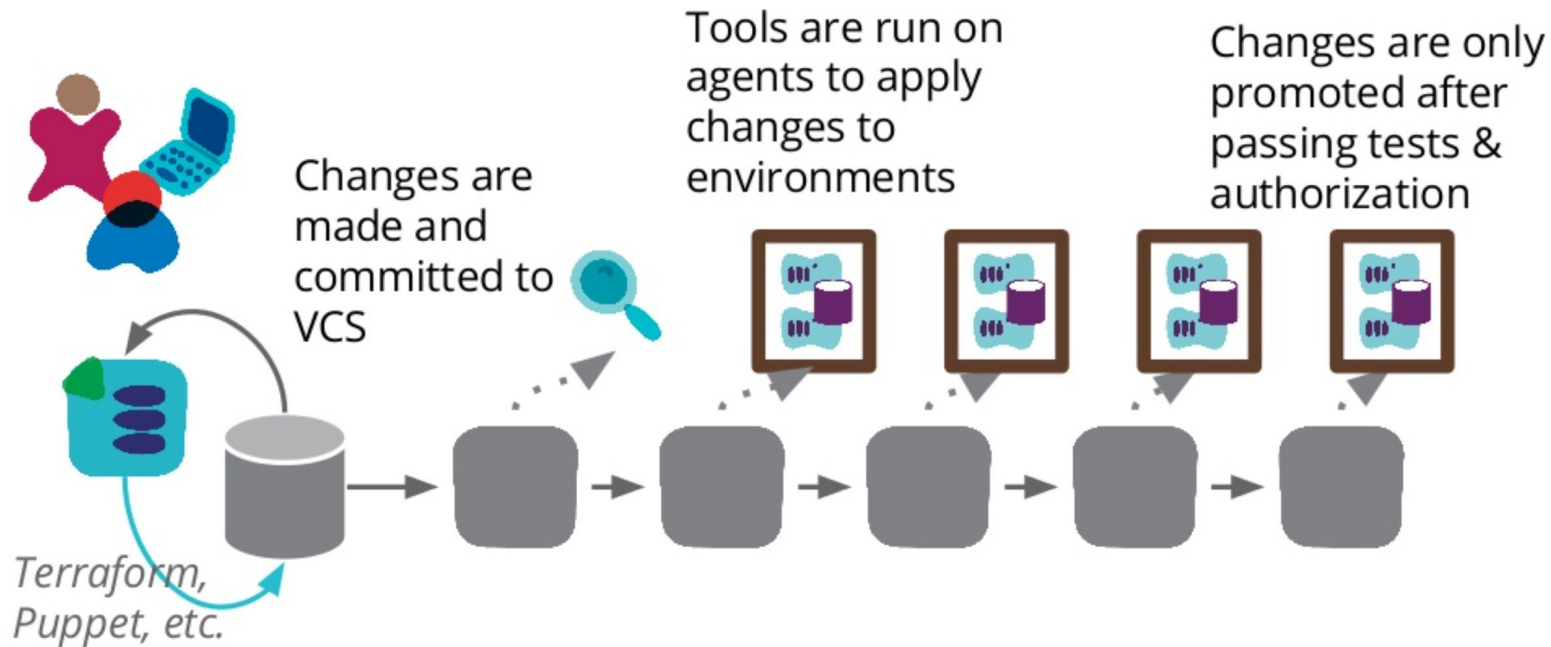
Запобігання DevOps



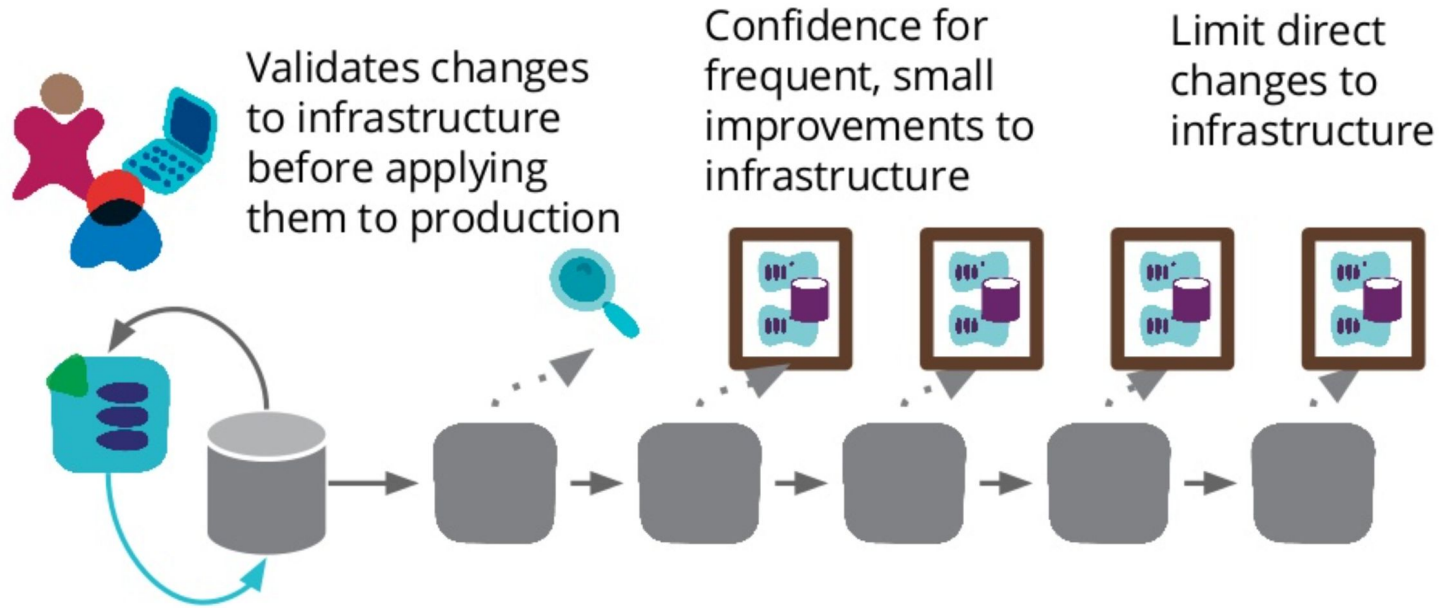


Потоки обработки данных

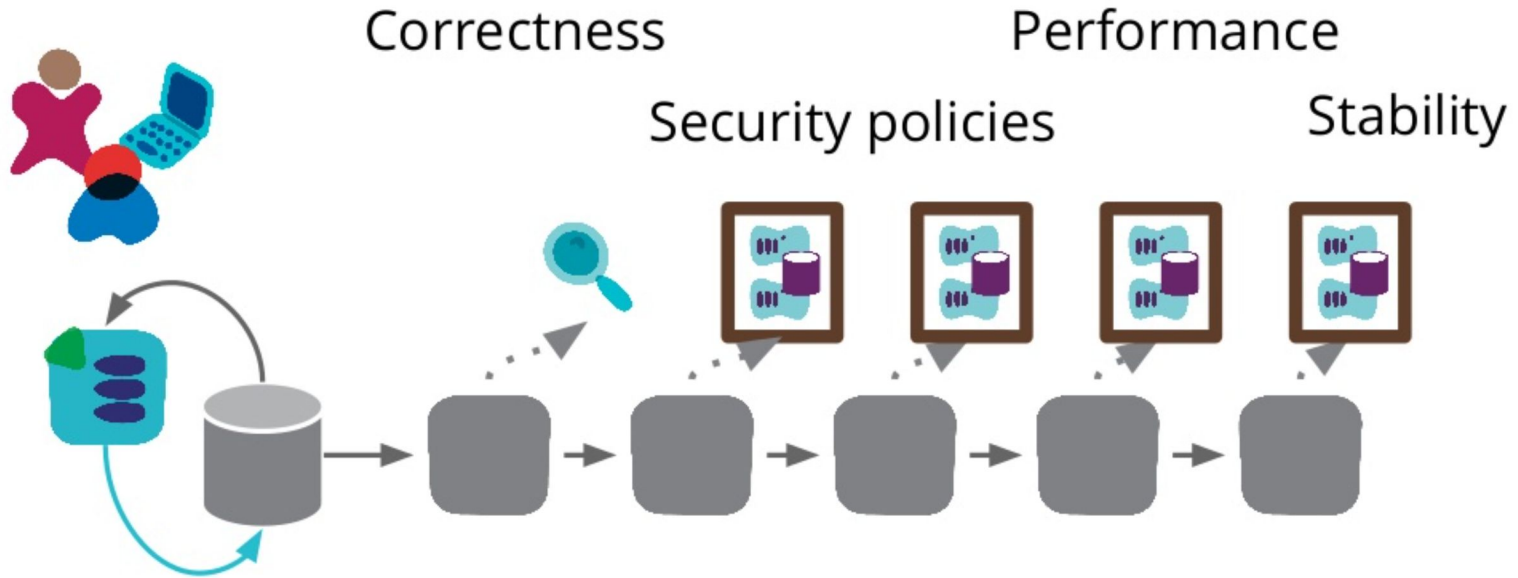
ЩО?



40MY?

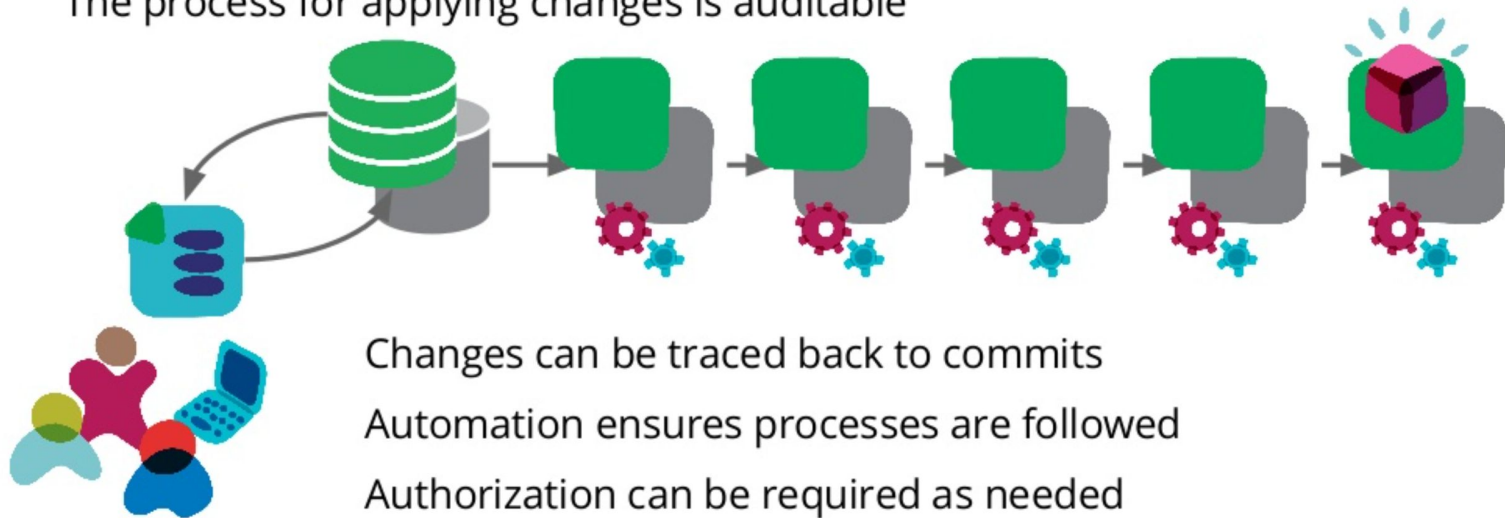


ТЕСТУВАННЯ



УПРАВЛІННЯ

The process for applying changes is auditable





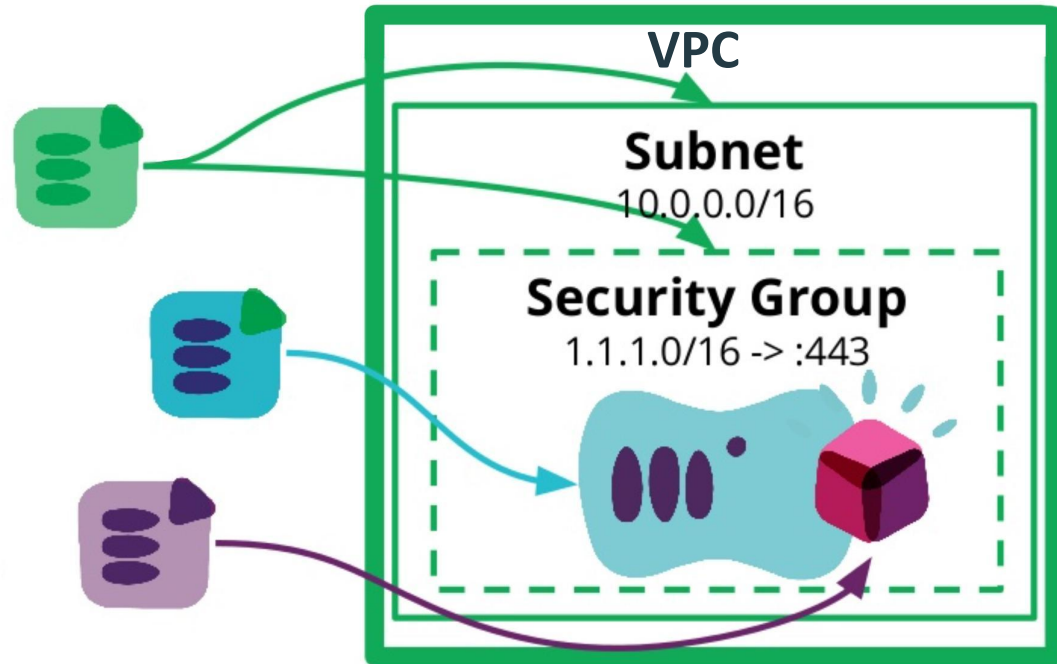
Простий приклад середовища

ВИЗНАЧЕННЯ ПРОСТОГО СЕРЕДОВИЩА

TERRAFORM FILE
Environment structure

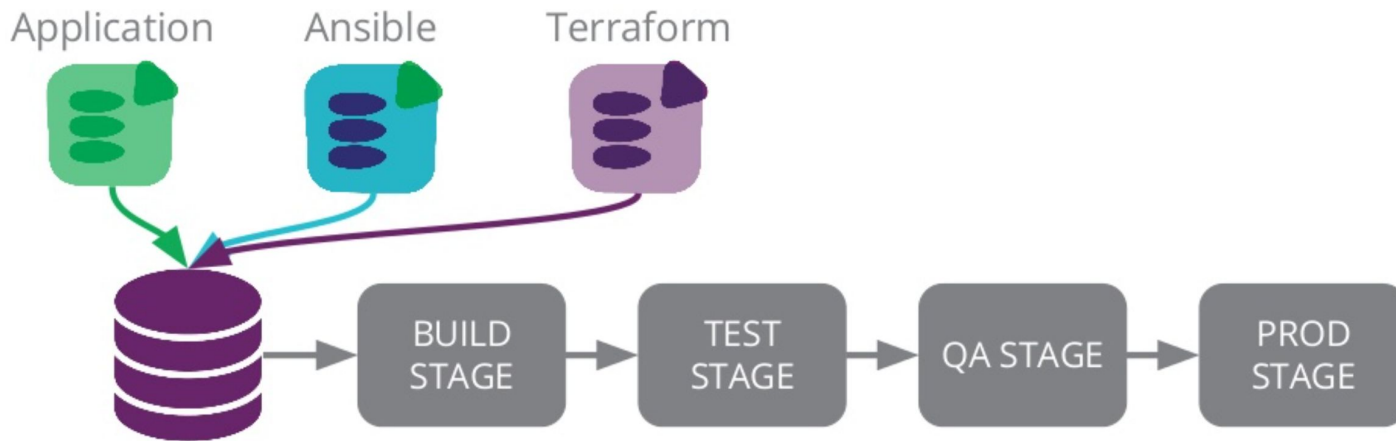
ANSIBLE PLAYBOOK
Server configuration

APPLICATION SOURCE
Deployable application



ПРОСТА КОНСТРУКЦІЯ ПОТОКУ ОБРОБКИ ДАНИХ

Deploy application, configuration, and infrastructure





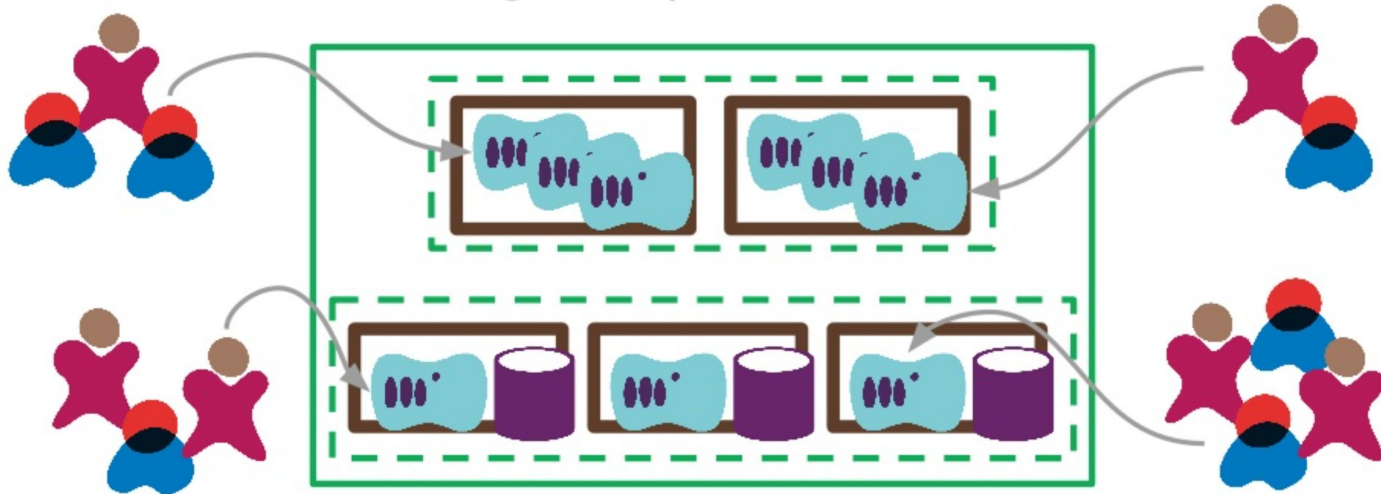
Масштабування

ПІДГОТУЙТЕ ДИЗАЙН ІНФРАСТРУКТУРИ ДО КОМАНД

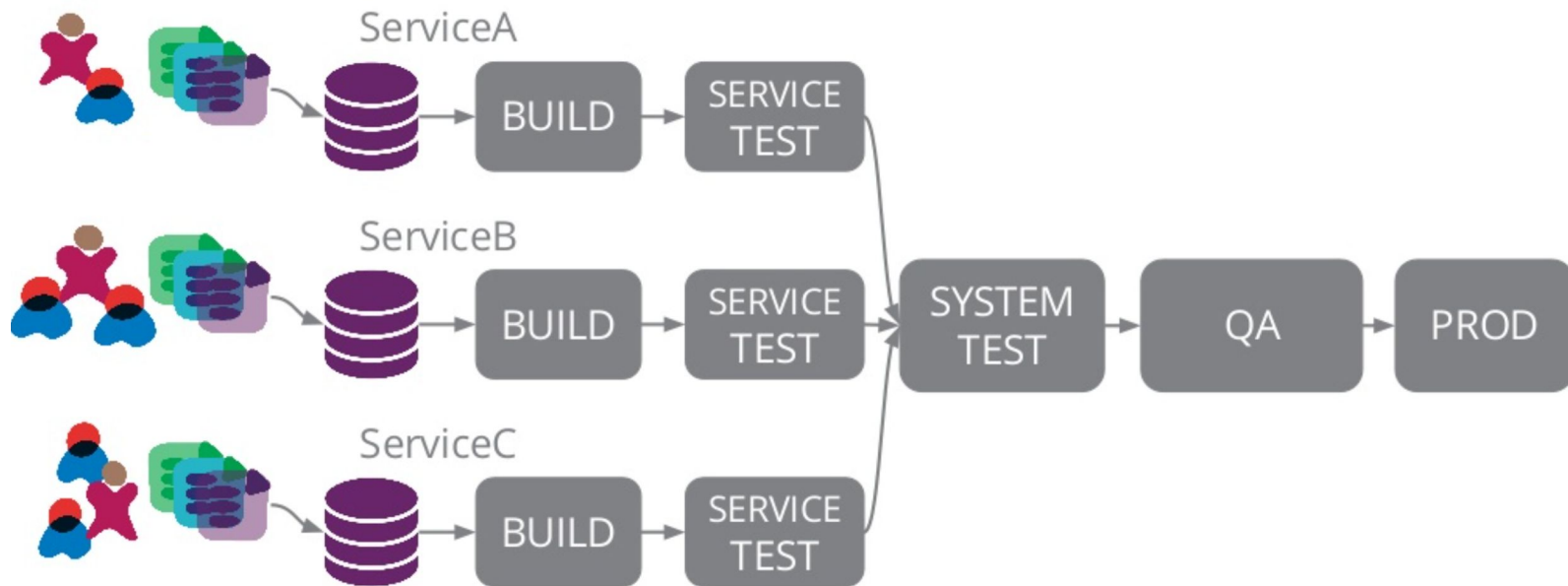
Переконайтесь, що команди можуть легко та безпечно вносити необхідні зміни

СКЛАДНЕ СЕРЕДОВИЩЕ

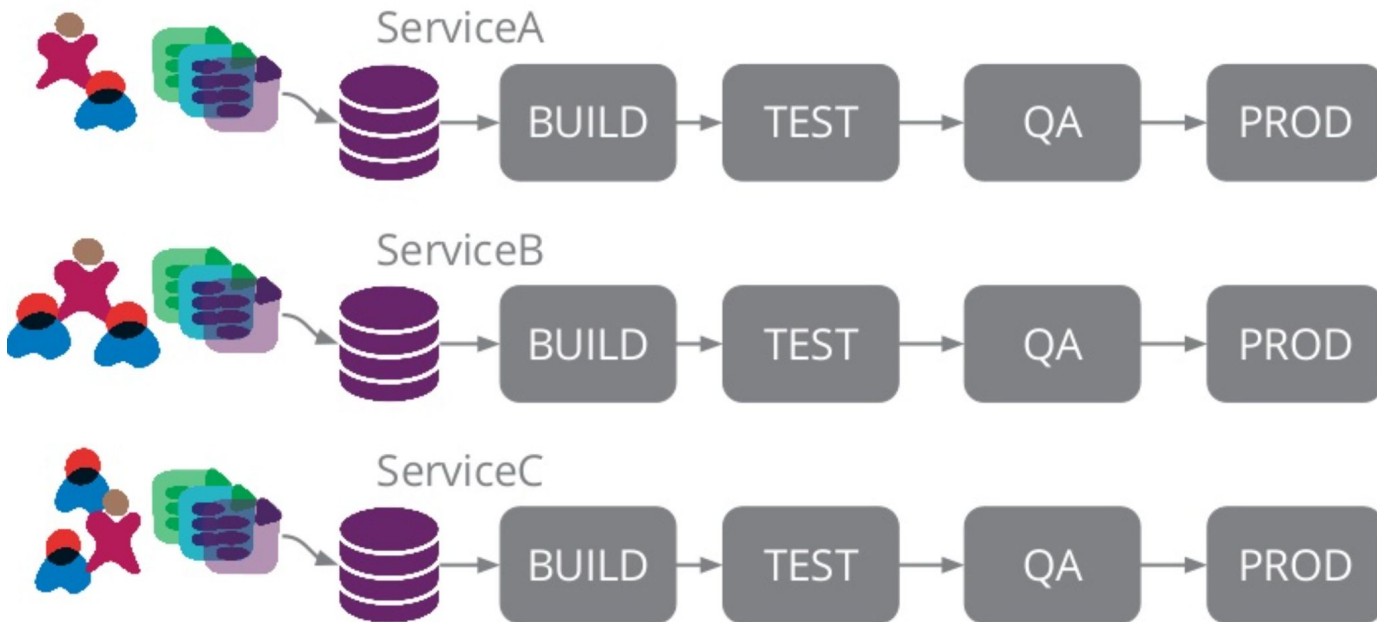
Infrastructure involving multiple teams



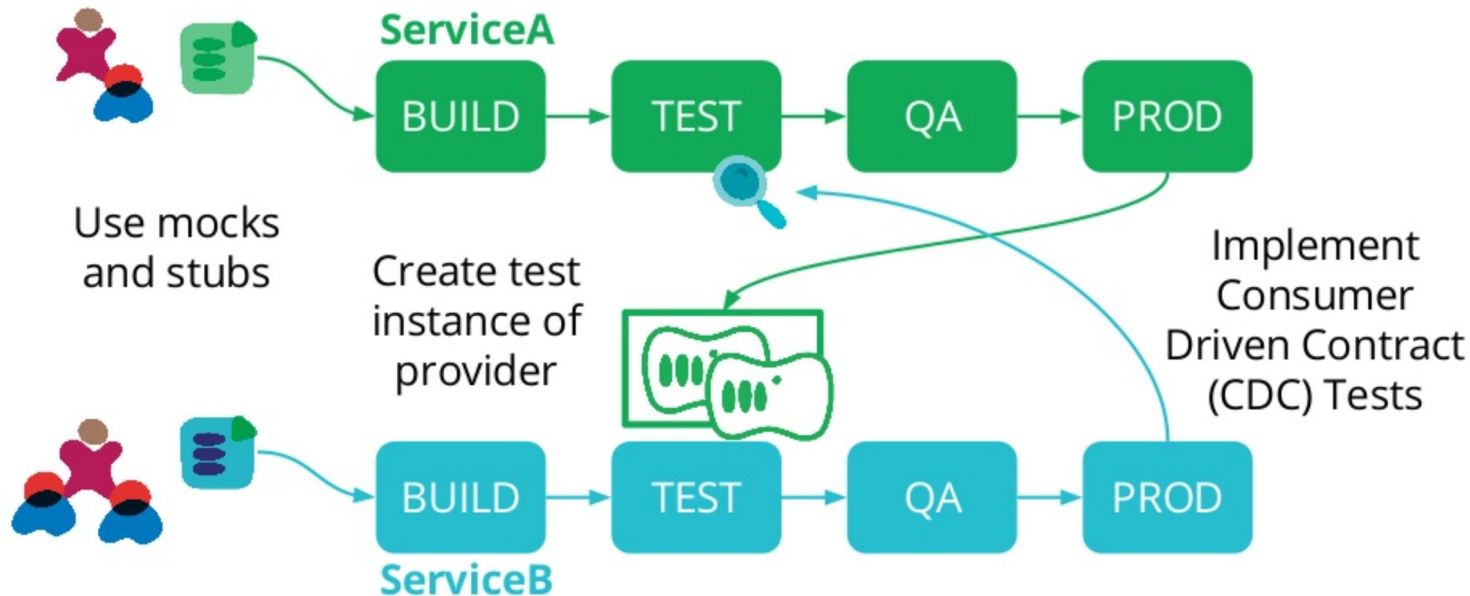
ВЕНТИЛЯТОРНИЙ ПОТІК ОБРОБКИ ДАНИХ



РОЗ'ЄДНАНІ ПОТОКИ ОБРОБКИ ДАНИХ

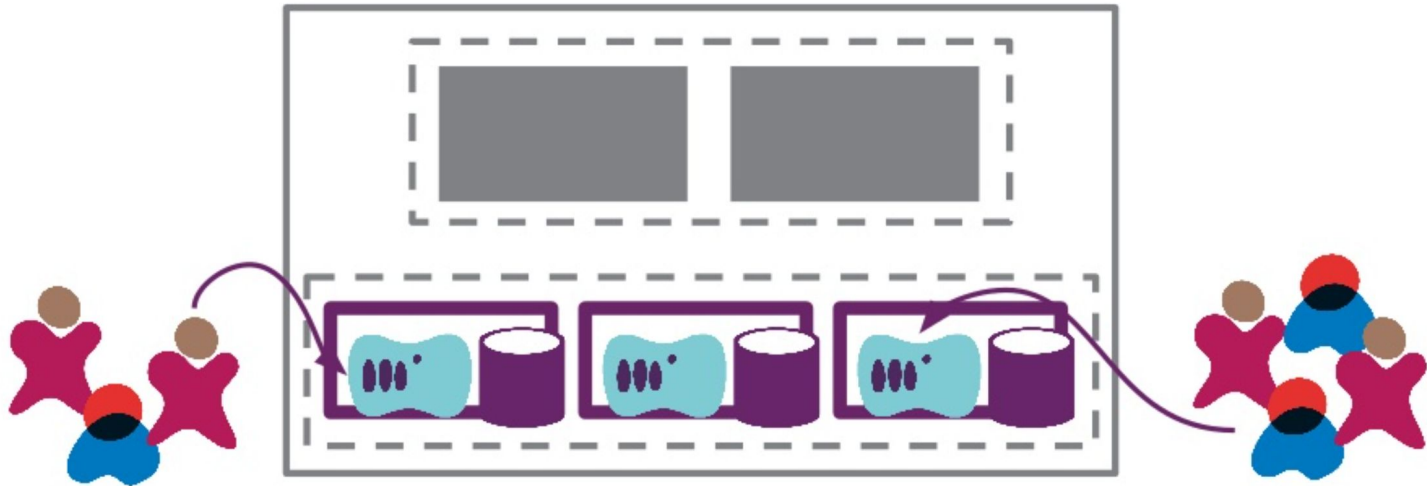


ЗАЛЕЖНОСТІ



ПРОБЛЕМА: ДУБЛІКАТ

Multiple teams using similar systems, e.g. database clusters



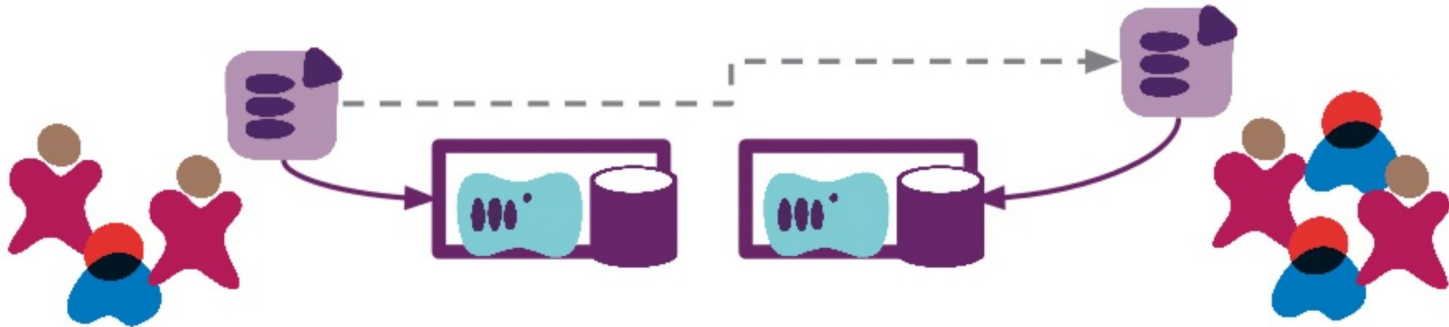
ПОВТОРНЕ ВИКОРИСТАННЯ ШЛЯХОМ РОЗГАЛУЖЕННЯ ВИЗНАЧЕНЬ

Disadvantages:

- Divergence and Inconsistency

Advantages:

- Avoid tight coupling
- Handles diverse requirements



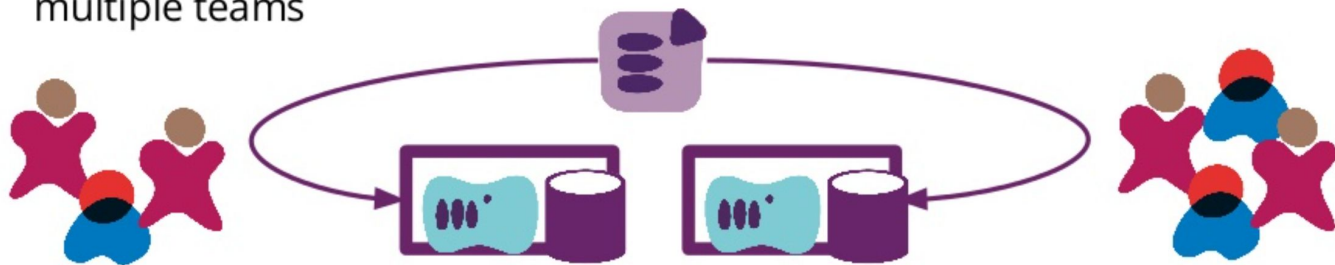
ПОВТОРНЕ ВИКОРИСТАННЯ З БІБЛІОТЕКАМИ ВИЗНАЧЕНЬ

Challenges:

- Avoid tight coupling, so teams aren't blocked when making changes
- Ownership of code shared by multiple teams

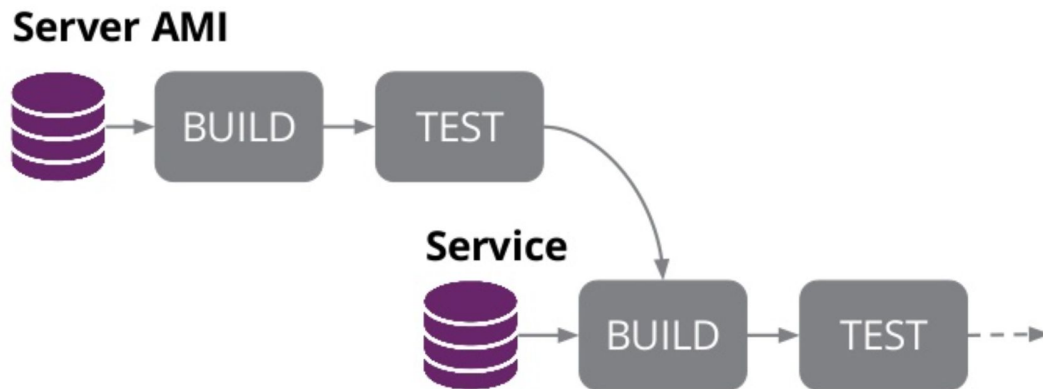
Guidance:

- Use separate pipelines for each
- Use CDC & other dependency testing strategies

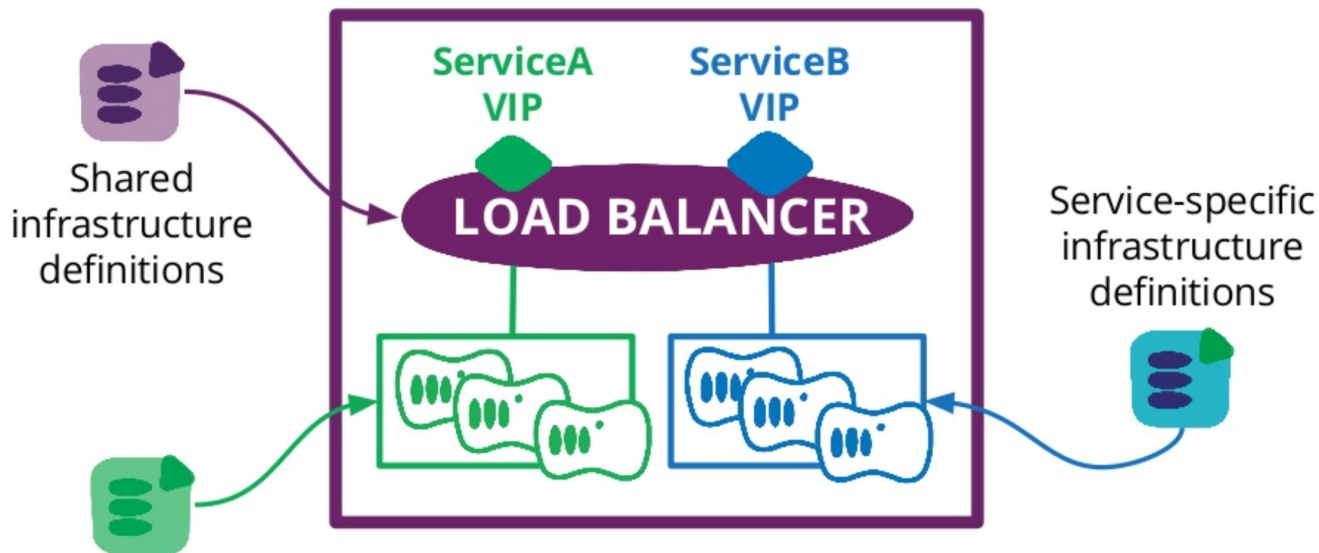


БІБЛІОТЕЧНИЙ ПОТІК ОБРОБКИ ДАНИХ

Test shared definitions before pulling them into dependent pipelines

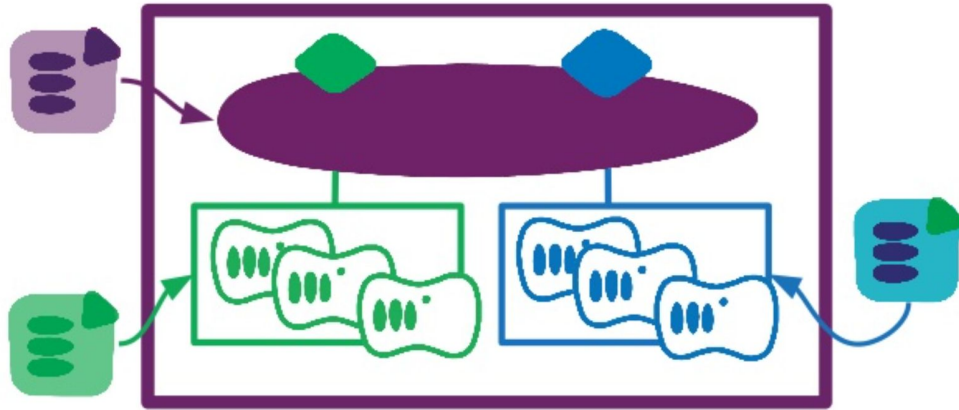


ПРОБЛЕМА: СПІЛЬНІ ЕЛЕМЕНТИ



ОБМІН ЕЛЕМЕНТАМИ

Avoid monoliths - optimize to simplify making changes





Інфраструктура як код (IaC) з Terraform і Ansible

Інфраструктура як код

Опишіть усі компоненти інфраструктури текстом

- Віртуальні машини
- Мережі
- Брандмауери, балансувальники навантаження...
- На завершення: повний комплекс послуг ...

Інфраструктура як код

- Використовуйте інструменти для створення та налаштування фактичної інфраструктури на основі IaaS
- напр. Terraform, Salt Cloud, Ansible, Openstack Heat

Переваги:

- Відтворювана інфраструктура
- Версійна інфраструктура
- Перевірена інфраструктура
- Задokumentована інфраструктура
- Зміни в інфраструктурі стають помітними
- Багаторазова інфраструктура

Інфраструктура як код

Коротко:

- Розглядайте інфраструктуру як дані (текст) і використовуйте ті самі методи, що й розробники програмного забезпечення:
- VCS: підтримуйте версії визначень інфраструктури
- TDD: перевірити інфраструктуру за допомогою `serverspec`, `testinfra`
- CI/CD: зміни інфраструктури проходять через конвеєр CI/CD

Інфраструктура як код

Деякі виклики:

- Як безпечно працювати з маркерами доступу до API в команді?
- Страх автоматизації
 - a. Немає довіри до засобів автоматизації, т.к
 - b. сервери не були налаштовані узгоджено, оскільки
 - c. автоматизація не запускається часто ... перейдіть до 1

Інфраструктура як код

Шаблони віртуальних машин:

- створювати нову інфраструктуру, використовуючи попередньо створені образи
- автоматизувати процес будівництва: напр. за допомогою пакувальника від HashiCorp

Будівництво інфраструктури за допомогою Terraform

- DSL (Domain Specific Language) для створення інфраструктури
- Terraform — це інструмент підготовки, а не інструмент керування конфігурацією
- Terraform використовує декларативний синтаксис
- Terraform може вносити поступові зміни в існуючу інфраструктуру

Будівництво інфраструктури за допомогою Terraform

Поступова зміна потребує інформації про стан:

- Terraform створює файл стану: terraform.tfstate

Як поділитися цією інформацією про стан між членами команди?

- Git, Amazon S3, Openstack Swift тощо, консул

Визначення ресурсу для віртуальної

```
38 resource "openstack_compute_instance_v2" "web_1" {
39
40     name = "web_1"
41     flavor_name = "m1.small"
42     key_pair = "sbeffa"
43
44     network {
45         uuid = "${openstack_networking_network_v2.private_ubuntu.id}"
46     }
47
48     user_data = "${template_file.web_1.rendered}"
49     floating_ip = "192.168.122.250"
50     image_id = "e4ce4179-9971-40f5-9c24-1230538ef82e"
51
52 }
```

- Terraform шукає файл main.tf у поточному робочому каталозі, головній точці входу
- Модуляризація підтримується

Налаштування інфраструктури за допомогою Ansible

- Ansible — це інструмент керування конфігурацією
- Відрізнити керування конфігурацією від ініціалізації не так просто
- Може також використовуватися для керування інфраструктурою, напр. модулі `ansible os_server` або `ec2`
- Але не має способу зберігати стан інфраструктури, як це робить Terraform

Тому використовуємо

- Terraform для побудови (створення) інфраструктури
- Ansible для налаштування інфраструктури

Налаштування інфраструктури за допомогою Ansible

Як склеїти Terraform і Ansible ?

- Спеціальний оркестратор, який спочатку виконує Terraform і потім Ansible

Питання та відповіді